

Agentic AI Overview

Agentic AI

Goal-driven AI systems that can reason about a problem, plan multiple steps, select and use tools, execute action, observe the results and adapt its next action with limited human intervention.



Independent Task
completion



Dynamic Process +
Tool usage



Autonomous
Decision + Action



Objectives + tools
+ context



Autonomous Agents +
GenAI + Adapt



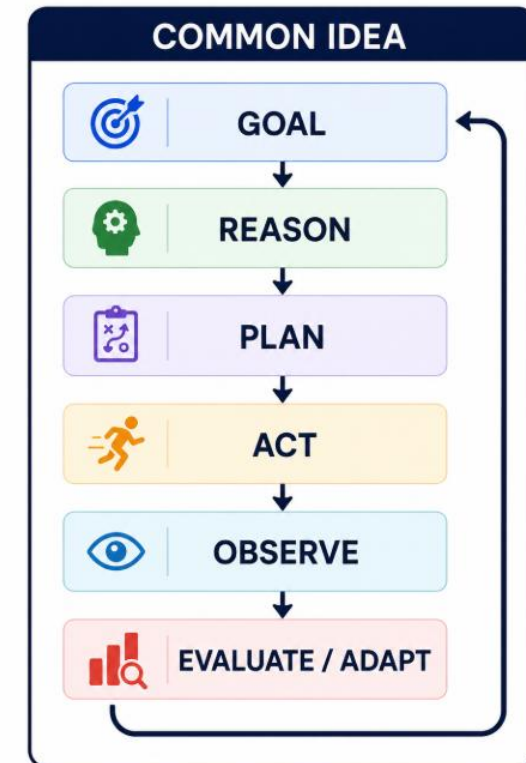
Reason + Plan +
Act + Tools



Goals + limited
supervision



Reasoning + Planning + tools +
Memory + Feedback



FOUNDATION

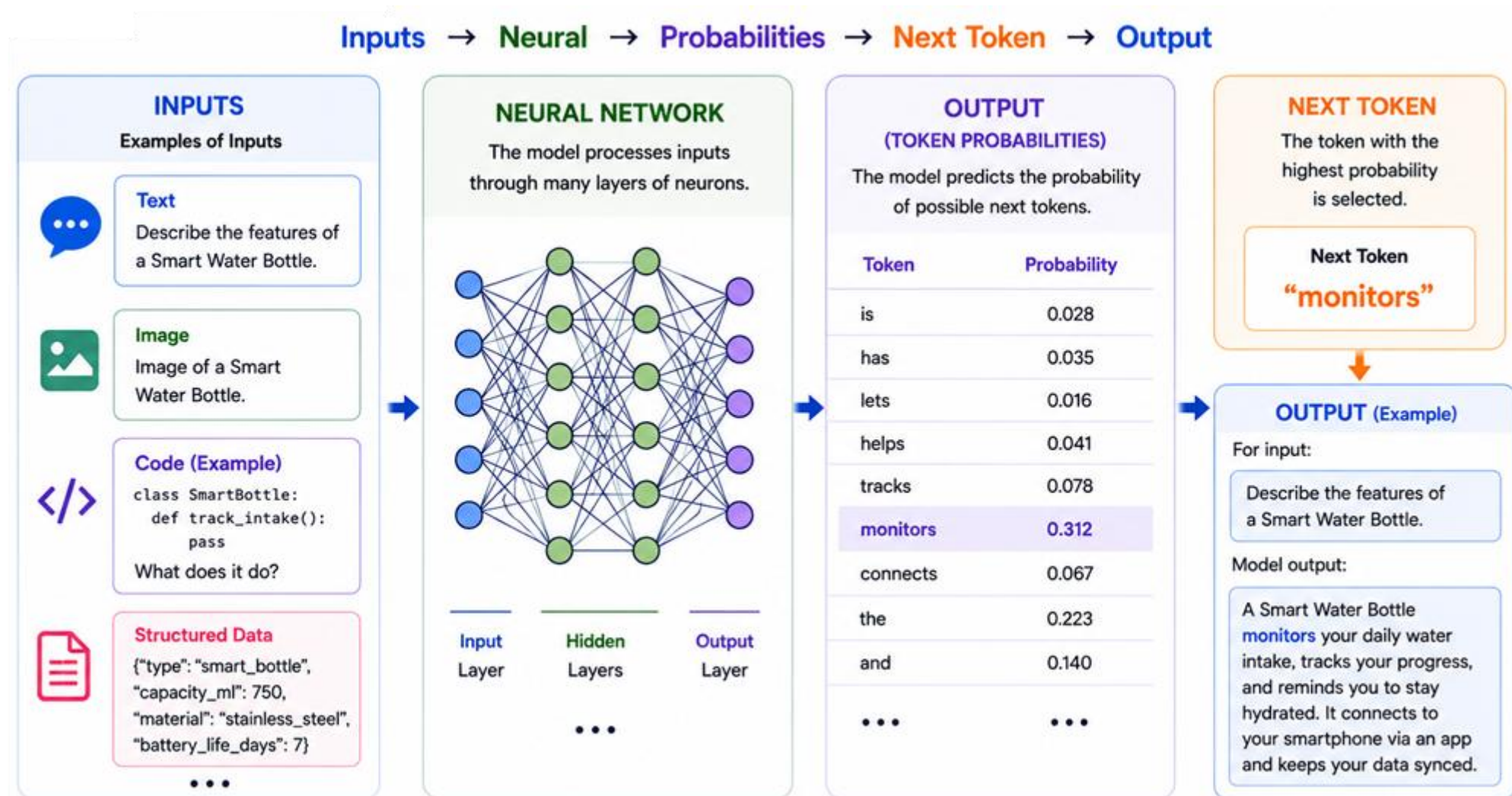
Building Blocks of an Agentic Systems

Core components that combine to create an intelligent agent.

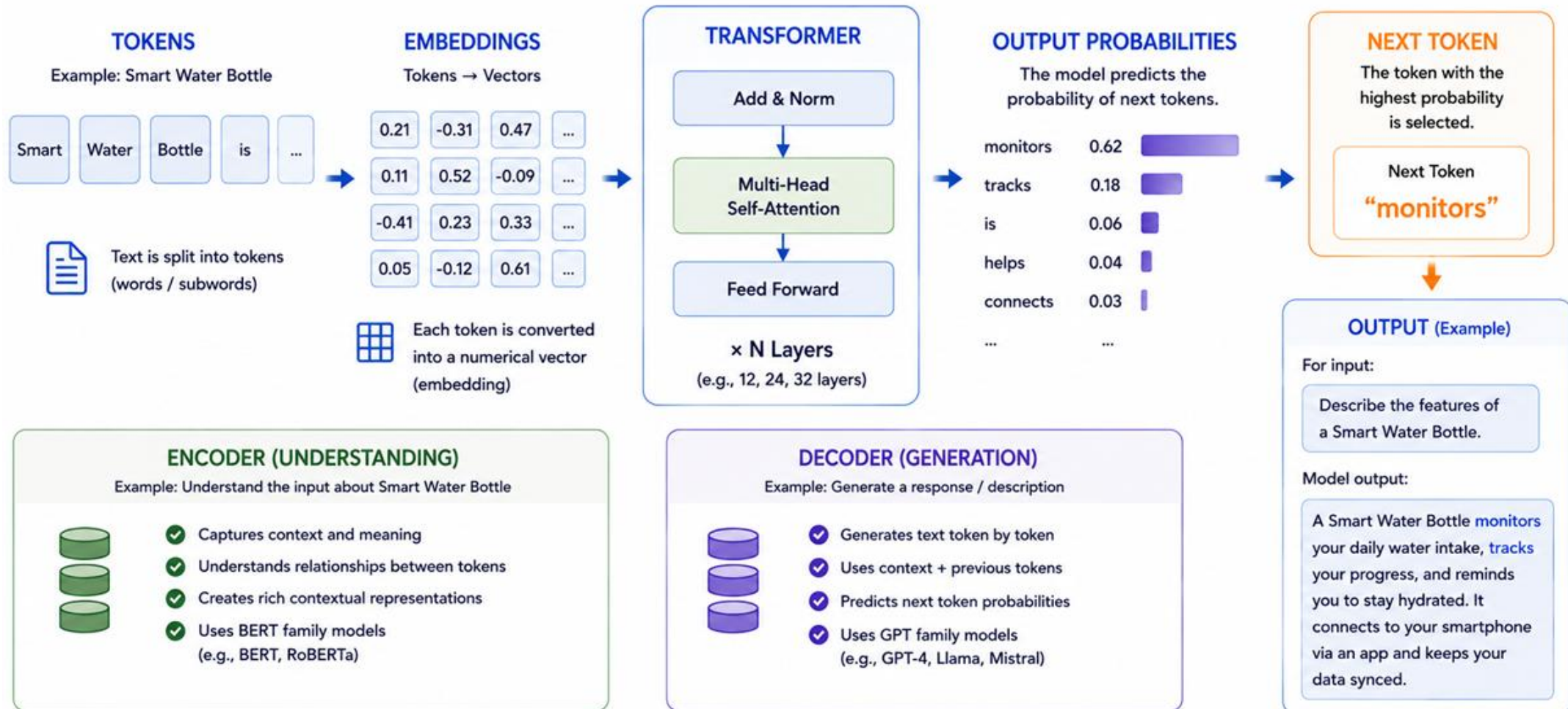
1. Model
2. LLM
3. Prompt Engineering
4. Context Engineering
5. RAG (Retrieval Augmented Generation)
6. Tool Calling
7. Agent Loop



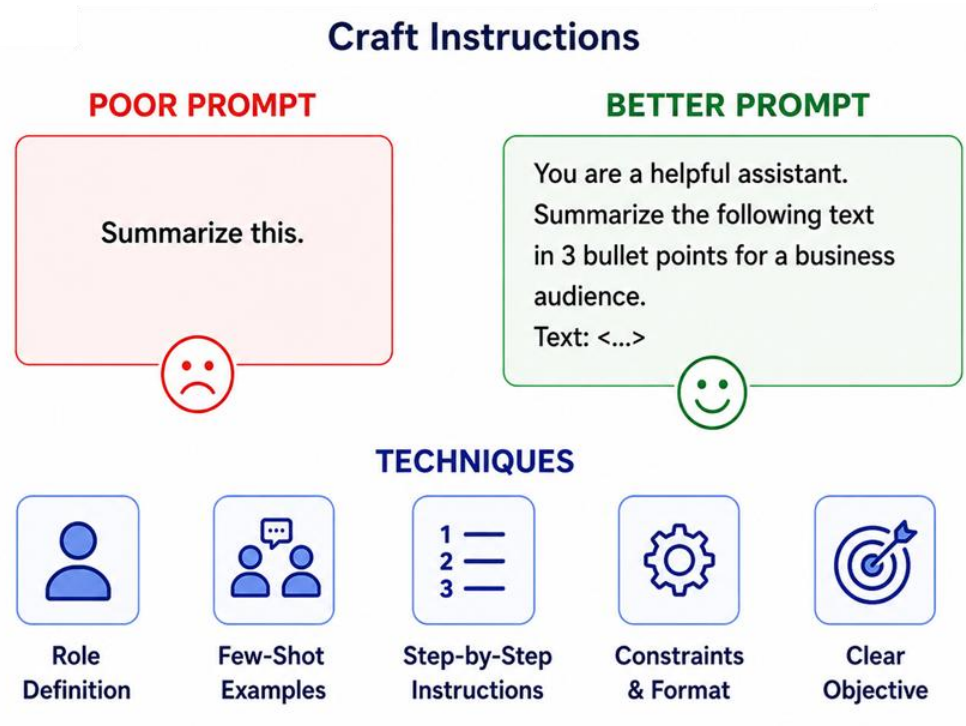
1. Model



2. Large Language Model



3. Prompt Engineering

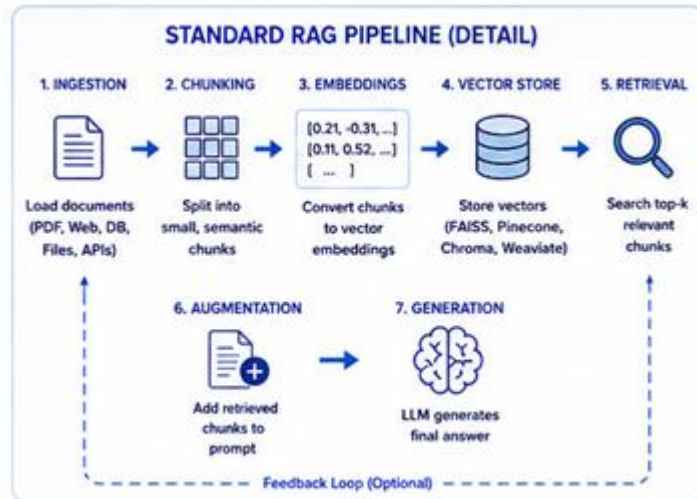
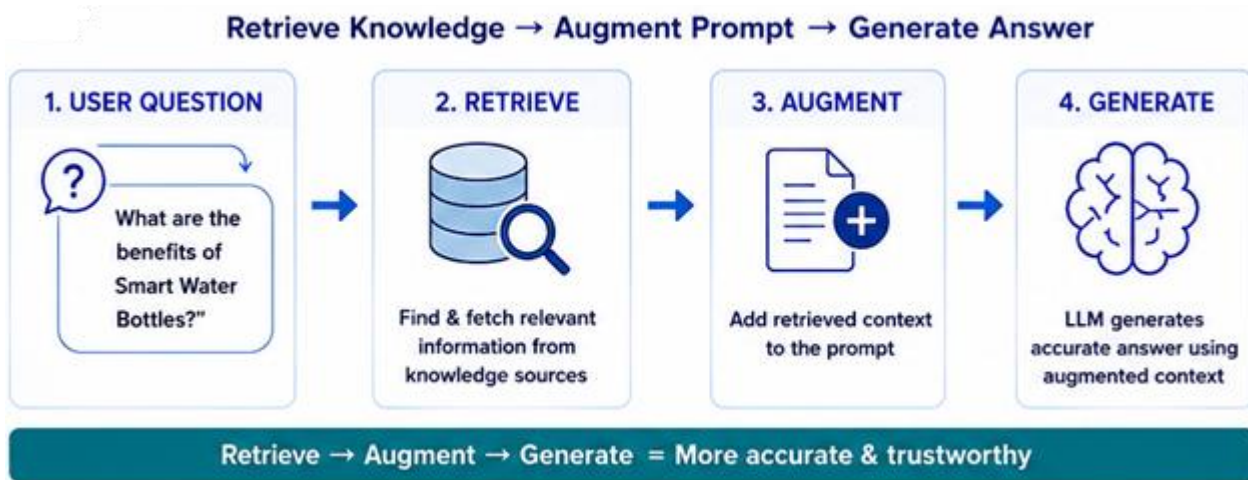


4. Context Engineering



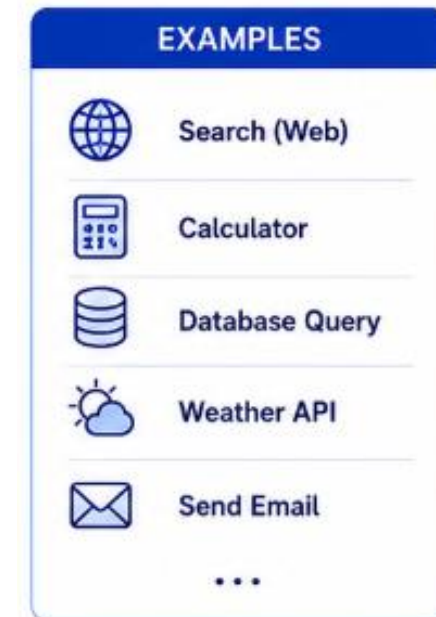
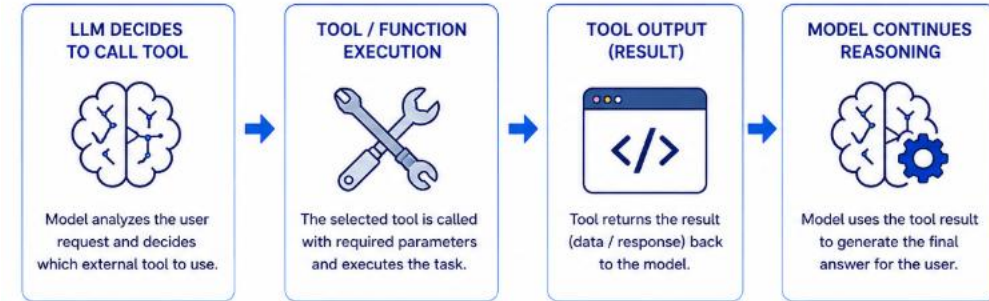
**Right Context
Better Understanding
Better Answer**

5. Retrieval Augmented generation (RAG)

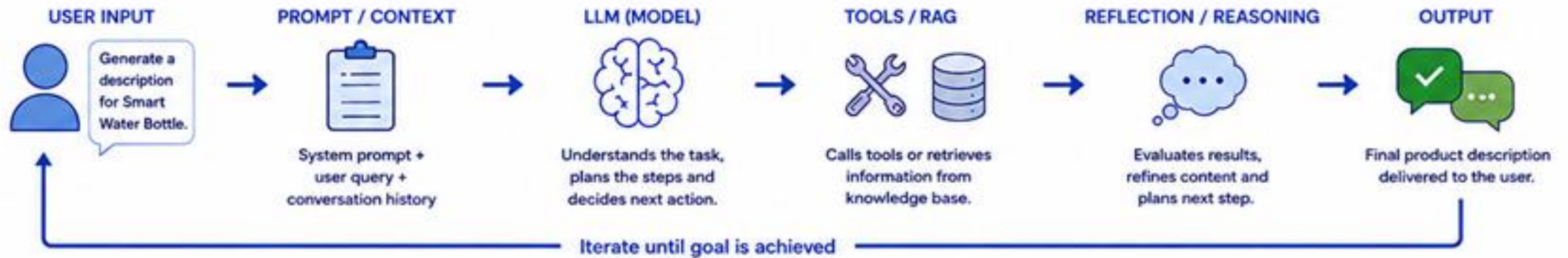


- More accurate and up to date
- Reduce hallucinations
- Use private/ domain specific data
- Cost efficient
- Scalable and modular

6. Tool Calling



7. Agent Loop



Why is Agentic AI

THE SHIFT: FROM TRADITIONAL AI TO AGENTIC AI

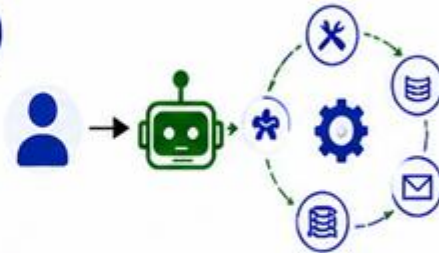
TRADITIONAL AI / LLM One-shot Response



- ✗ Single prompt → Single response
- ✗ Limited to model knowledge
- ✗ No memory of past actions
- ✗ Cannot take actions

VS.

AGENTIC AI SYSTEM Goal-Driven Execution



- ✓ Understands goal and breaks it down
- ✓ Plans and chooses the right tools
- ✓ Remembers context and learns
- ✓ Takes multiple actions to achieve the goal

MULTIPLE TOOL CALLING IN A SINGLE OPERATION

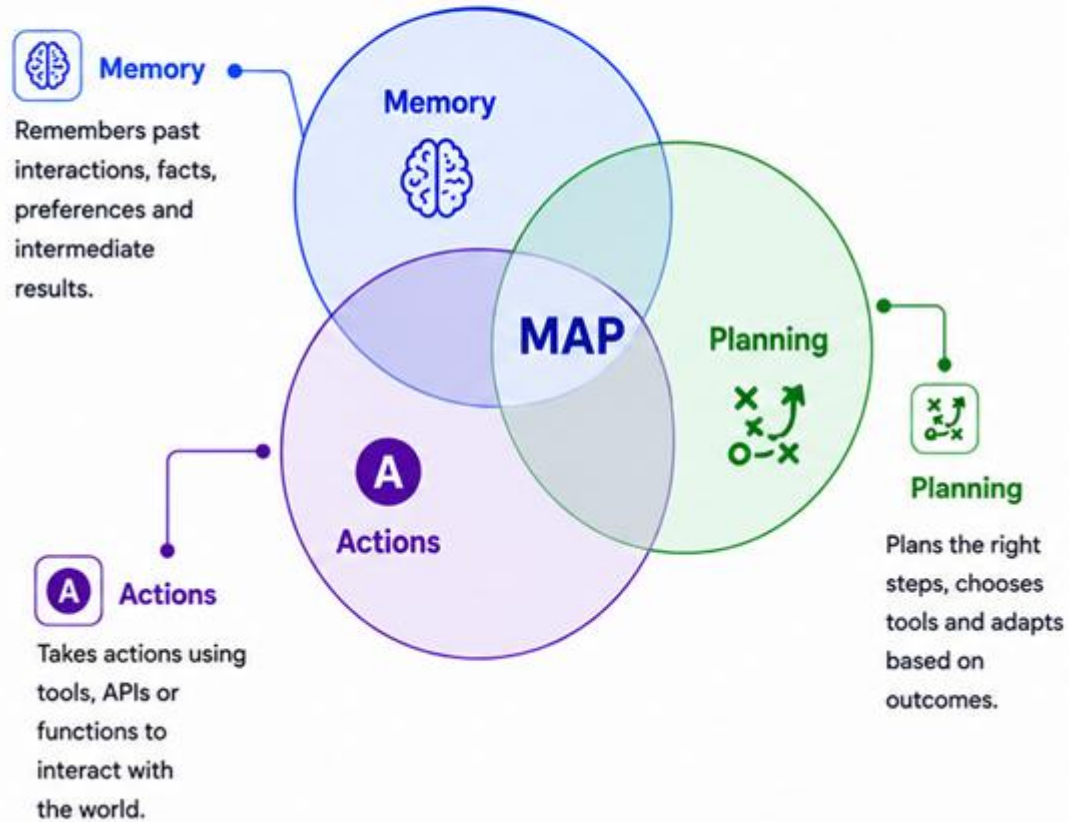
Agentic AI can call multiple tools (APIs, functions, systems) in sequence or in parallel to complete a task.



★ One goal → Many decisions → Many tool calls → Best outcome

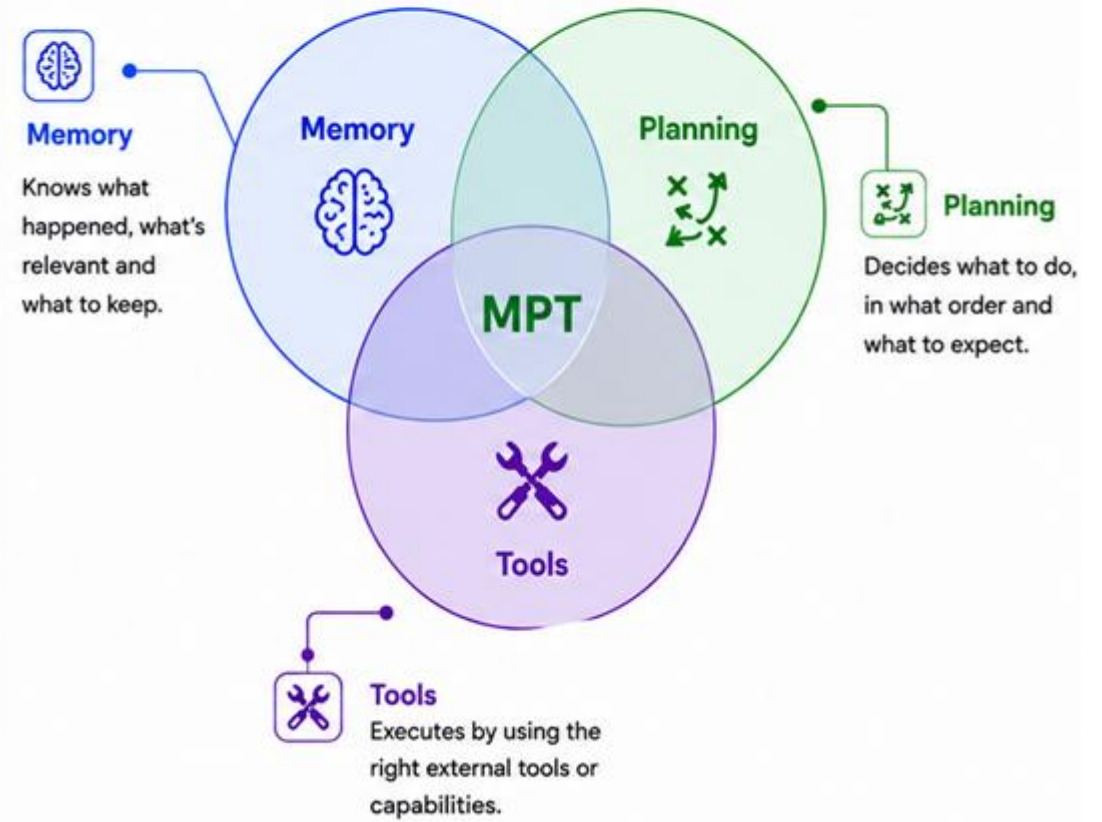
MAP

Memory, Actions, Planning



MPT

Memory, Planning and Tools





CREWAI

Role-based Multi-Agent Framework

CrewAI helps you build role-playing AI teams where agents collaborate to achieve a goal.

KEY FEATURES



- Role-based agents with specific goals and backstories



- Agents collaborate in a crew to complete tasks



- Built-in task delegation and execution flow



- Easy to get started for structured use cases

BEST FOR



Use cases where defined roles and collaboration between agents are key (e.g., research team, content creation, analyst crew).



AUTOGEN

Conversational Multi-Agent Framework

AutoGen enables LLM agents to converse, debate, plan and execute tasks through message passing.

KEY FEATURES



- Conversation-driven agent interaction



- Supports multi-agent chats, debate and collaboration



- Flexible patterns: conversation, hierarchical, group chat



- Great for exploring autonomous agent behaviors

BEST FOR



Research, prototyping and scenarios where agents need to discuss, debate or dynamically collaborate to solve problems.



LANGGRAPH

Stateful Agent Workflow Framework

LangGraph helps you build stateful, reliable and controllable agent workflows as graphs.

KEY FEATURES



- Graph-based workflows (nodes, edges, state) for full control



- State management across steps (intelligent memory)



- Conditional routing, loops, retries and human-in-the-loop



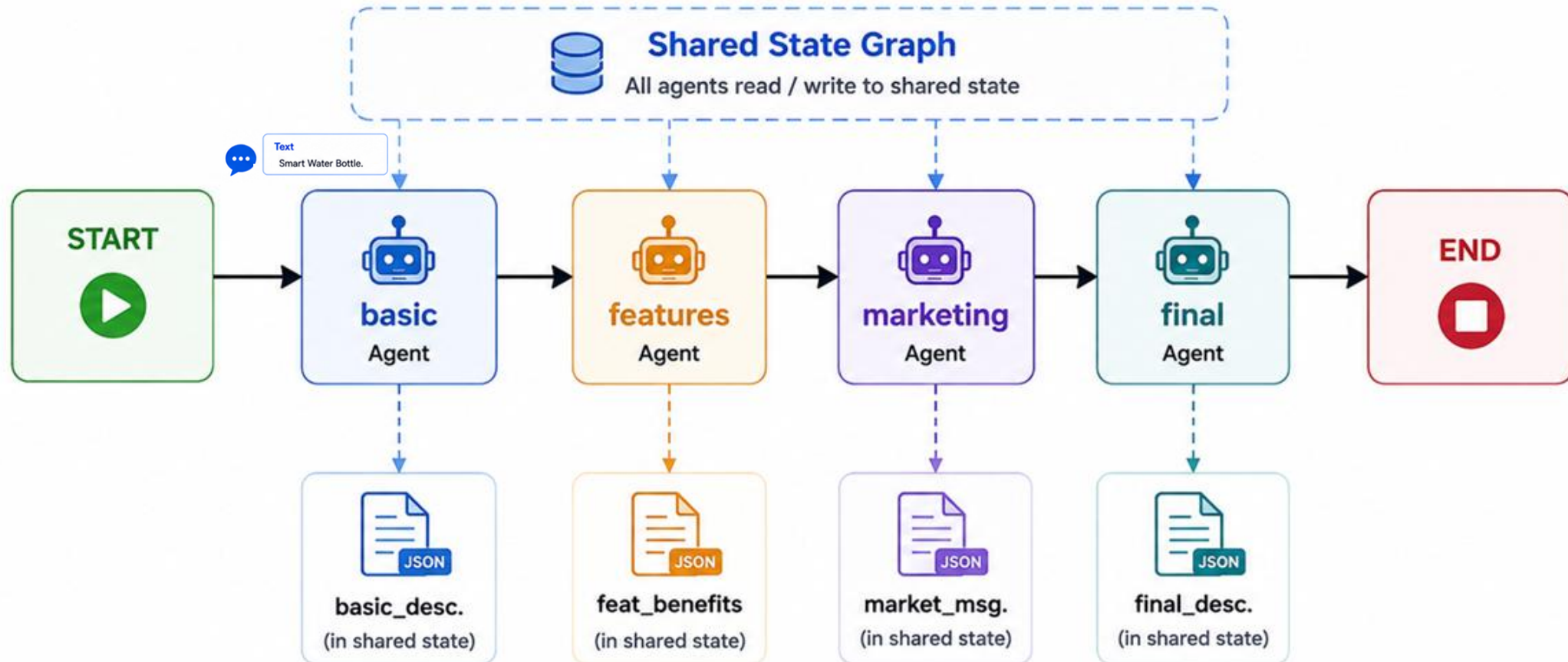
- Production-ready: durable execution, persistence, observability

BEST FOR



Complex, production-grade agent systems that require state, control, reliability and long-running workflows.

Workflow Concept 1 – Sequential Agent Graph



Shared State Mechanics in Sequential Agent

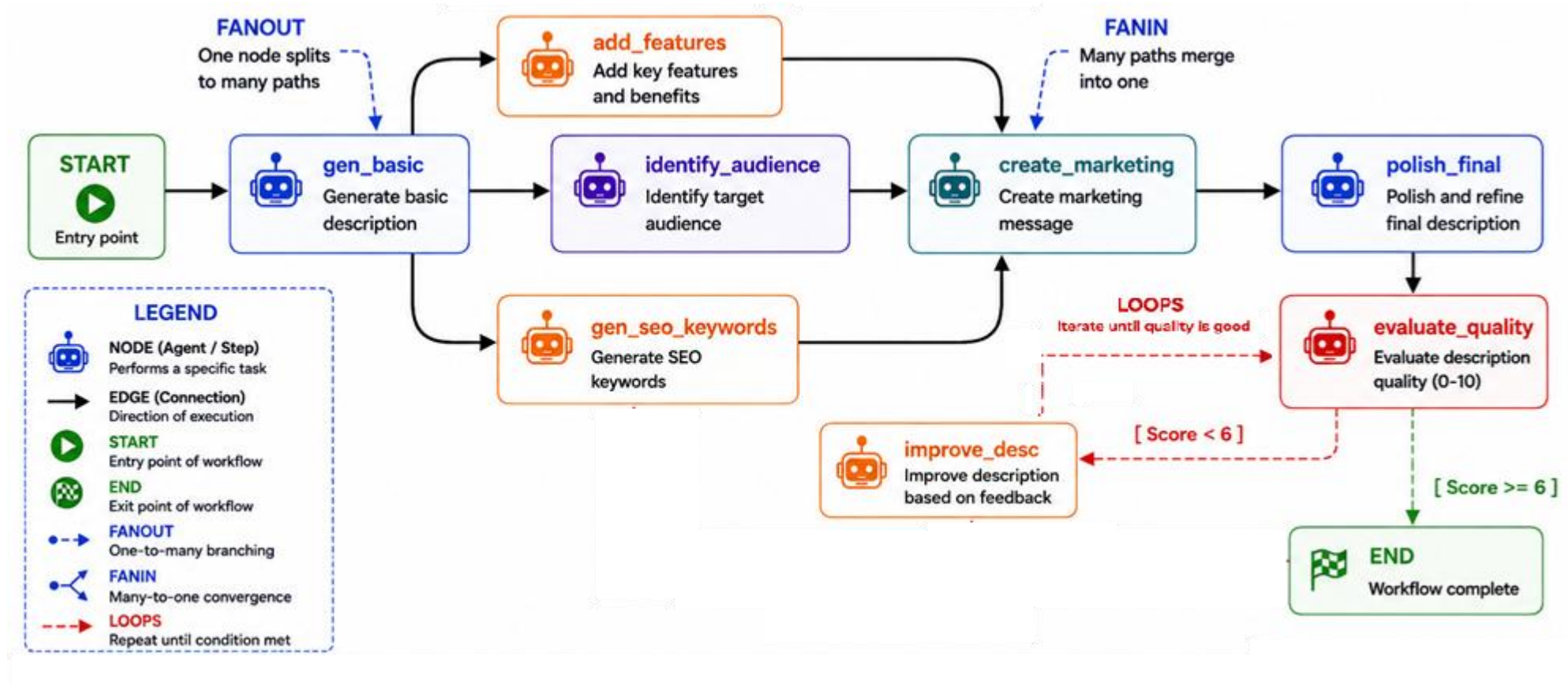
Agents read and write to the shared state object.
Each agent mutates the state by adding/ updating its part.



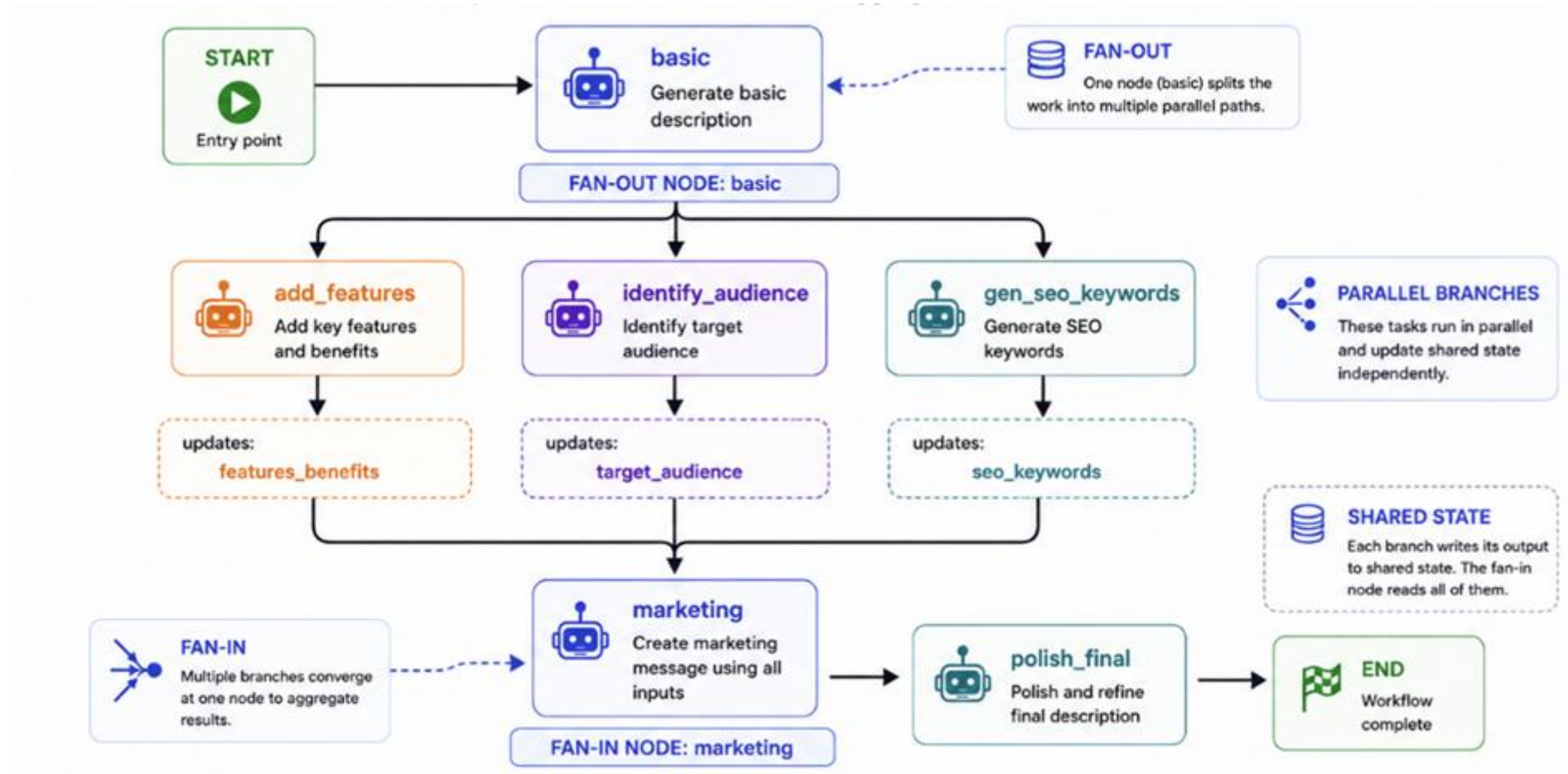
Shared state = Central JSON like object accessible to all agents.

Implementation

Workflow Concept 2 –Advanced Graph Topology

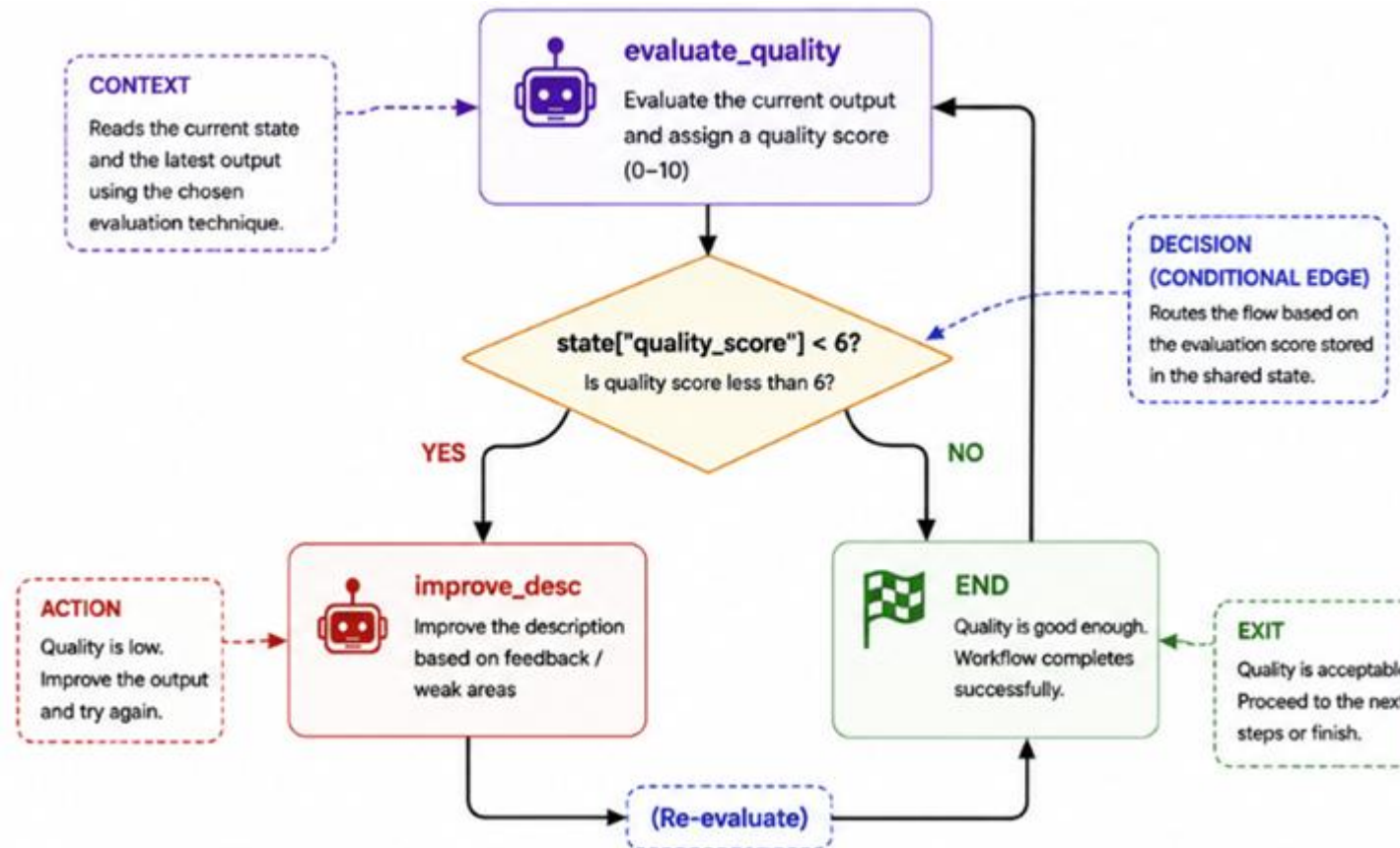


Parallel Branches & Aggregation Topology



Dynamic Control Flow –

Conditional Edges and Evaluation Loops



AVAILABLE EVALUATION TECHNIQUES

i In this project, we used `randint` generation just for demo.



1. LLM-as-a-Judge (Prompted Scoring)

Use an LLM with a rubric or prompt to score output (0–10) on quality, clarity, relevance, etc.



2. Embedding Similarity

Measure semantic similarity with reference or ground truth using embeddings (cosine similarity, etc.).



3. Rule-Based / Heuristic Checks

Check required keywords, length, format, structure, readability, domain rules, etc.



4. Model-Based Scores

Use classifiers or regression models trained to predict quality scores.



5. Human-in-the-Loop

Human reviewers provide scores / feedback for high-stakes or final validation.



6. Hybrid Scoring

Combine multiple signals (LLM score + similarity + rules + model score) for robust evaluation.

Implementation

Advantages & Disadvantages



LLM (Large Language Model)



Advantages

- Simple to use and quick to get started
- Great for natural language understanding and generation
- No orchestration setup required
- Low latency for single-turn tasks
- Cost-effective for small, simple tasks



Disadvantages

- No memory of past interactions (stateless)
- Limited to single-step tasks
- Can hallucinate or produce inconsistent outputs
- No built-in tool usage
- Not reliable for complex or multi-step tasks
- Hard to enforce strict output structure



WORKFLOW (Fixed Orchestration)



Advantages

- Deterministic and predictable execution
- Easy to debug, test and monitor
- Ensures correct order of steps
- Good for repeatable business processes
- Easier to handle errors at each step
- Scalable for well-defined pipelines



Disadvantages

- Rigid and less adaptable to change
- Hard to handle dynamic or open-ended goals
- Complex to maintain as steps grow
- Not intelligent – follows fixed rules
- Cannot decide next step on its own



AGENT (Goal-Driven & Adaptive)



Advantages

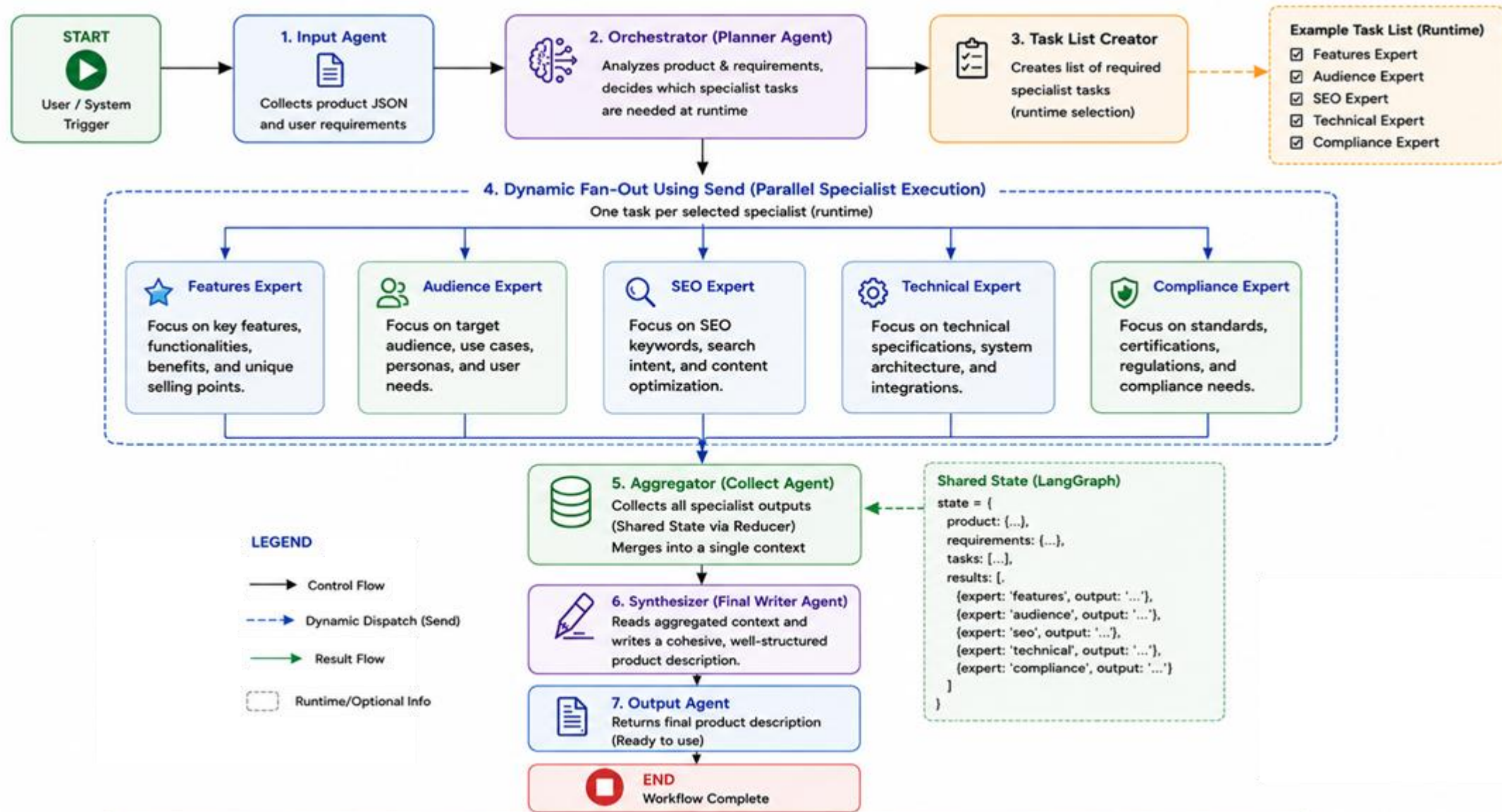
- Goal-driven and autonomous
- Adapts to changes and dynamic situations
- Can plan, reason and choose next steps
- Uses tools, memory and feedback
- Better for complex, open-ended tasks
- Improves over time with feedback/loops



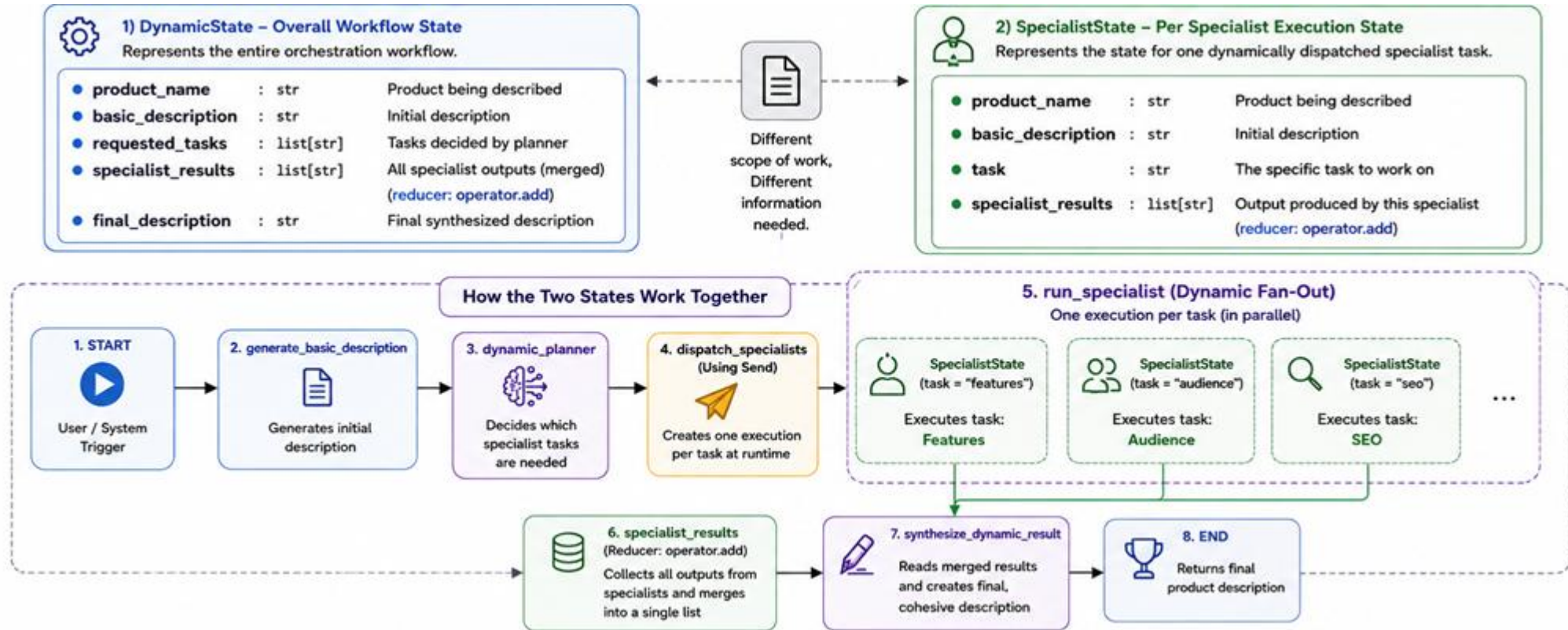
Disadvantages

- More complex to design, test and debug
- Higher cost (more LLM calls, tool usage)
- Requires guardrails and evaluations
- Harder to predict exact behavior
- Risk of infinite loops or wrong tool usage
- Needs good observability & monitoring

Dynamic Orchestration



Shared State Mechanics in Agentic Graph



Implementation

INDUSTRY	BUSINESS IMPACT – Key Factors Improved					
	 Efficiency	 Cost Savings	 Quality	 Customer Impact	 Speed	 Scalability
 Banking & Finance	 Automates analysis and reporting	 Reduces manual work & errors	 More accurate risk decisions	 Faster, 24/7 customer service	 Real-time decisions	 Handles large volumes easily
 Healthcare	 Reduces admin burden	 Lowers operational costs	 Improves diagnosis accuracy	 Better patient experience	 Faster clinical workflows	 Scales across institutions
 Retail & E-commerce	 Automates operations	 Optimizes inventory & pricing	 Better recommendations & offers	 Personalized experience	 Responds in real time	 Supports millions of customers
 Manufacturing	 Minimizes downtime & waste	 Reduces maintenance & operating costs	 Improves product quality	 Ensures reliable delivery	 Faster issue resolution	 Scales across plants & lines
 IT & Software	 Automates repetitive tasks	 Reduces engineering & support costs	 Fewer bugs & incidents	 Better internal user experience	 Faster releases & resolution	 Handles large codebases & users
 Education	 Automates content & grading	 Reduces admin workload	 Improves learning outcomes	 Better student engagement	 Instant support & feedback	 Supports large student numbers

Thank You